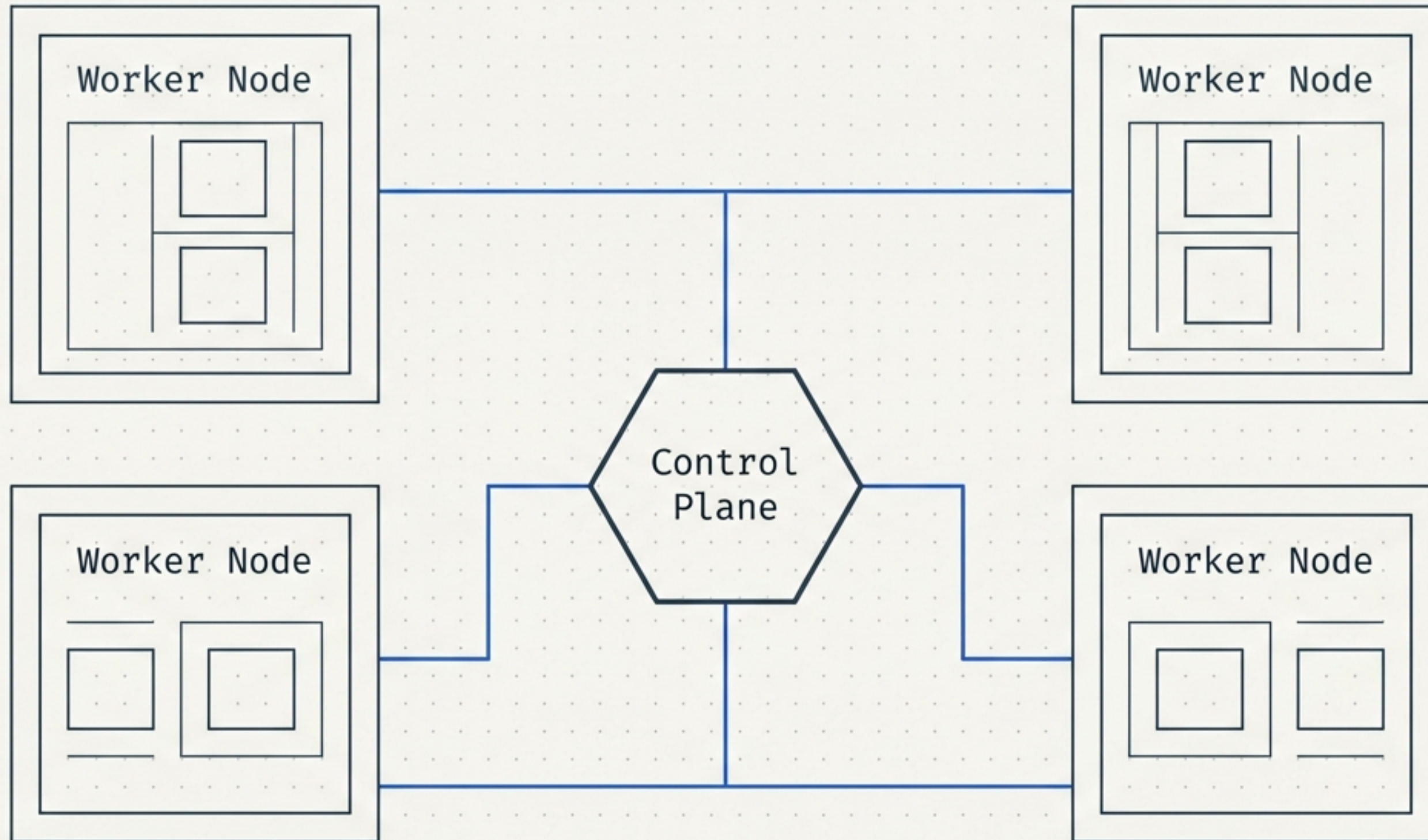




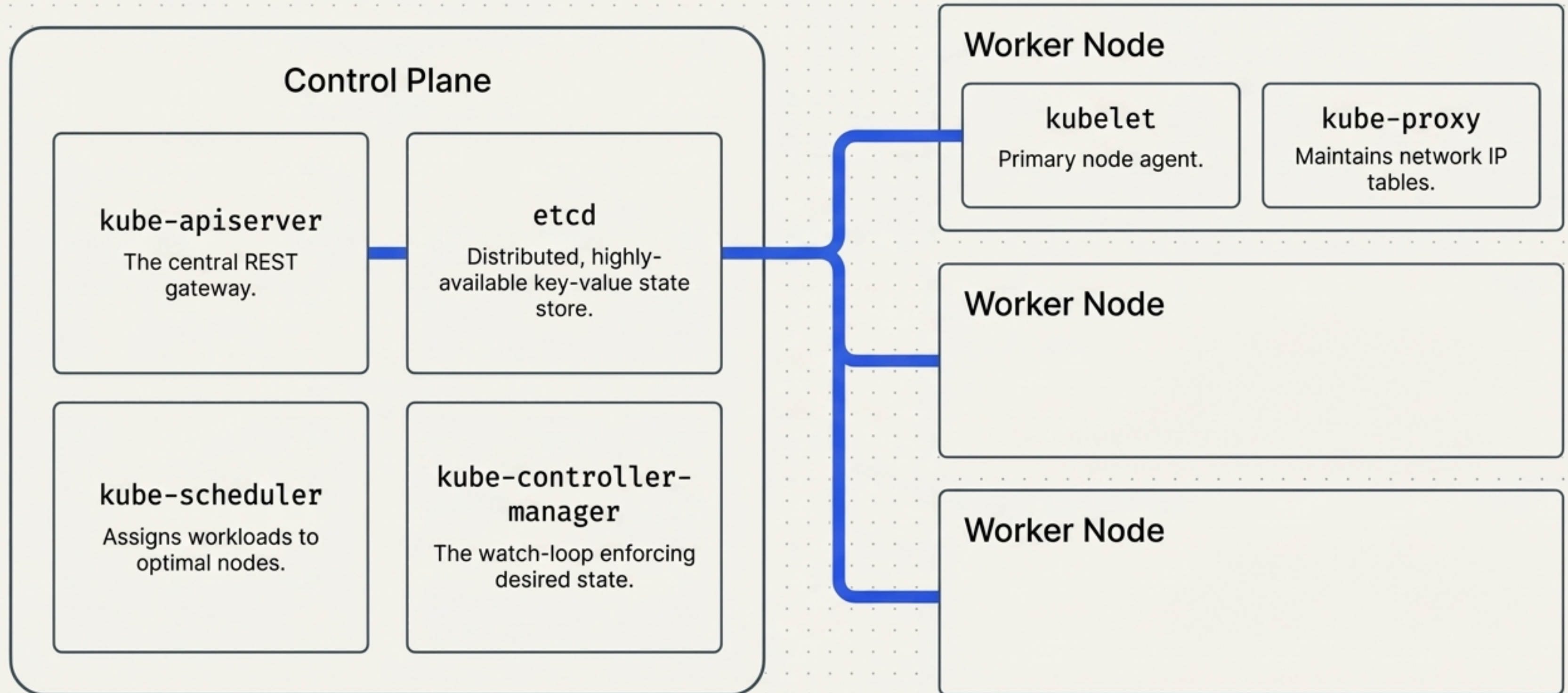
The Kubernetes Architect's Blueprint

A visual primer on cloud-native primitives, cluster networking, and microservices deployment.

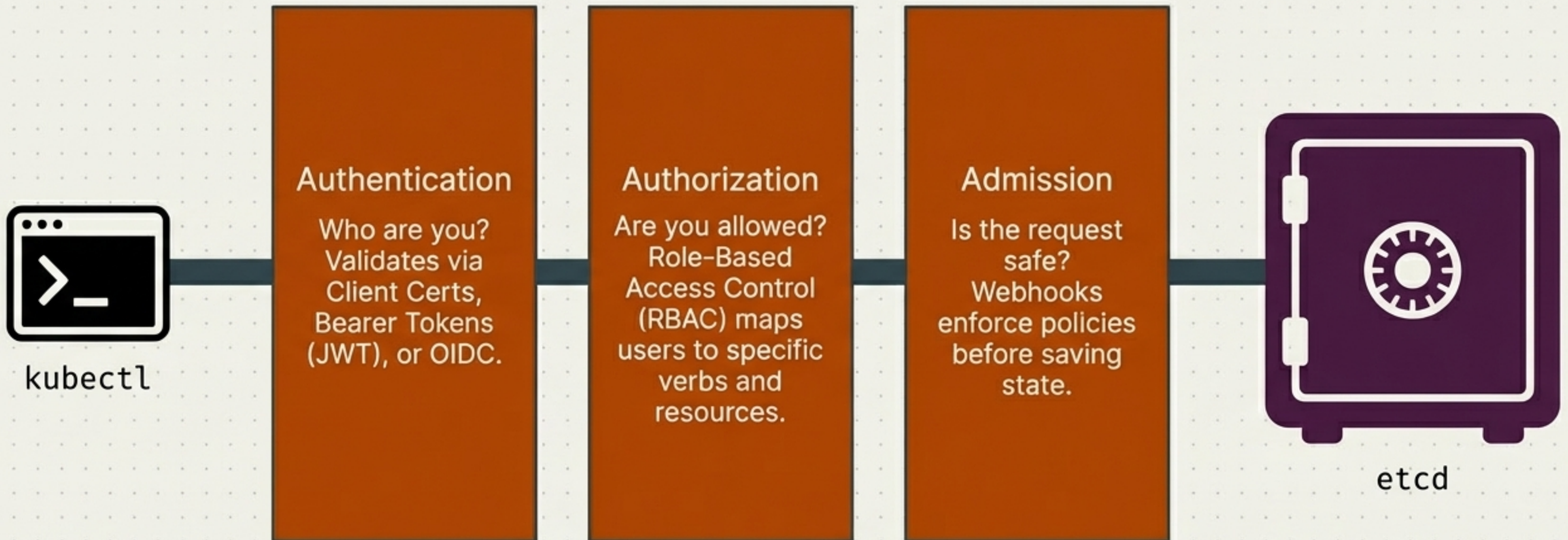
The Evolution of Application Infrastructure

	Bare Metal	Virtualization	Cloud	Containers
Resource Efficiency	Low (20-30% utilized)	Medium (Sliced CPU/RAM)	Medium	High (Shared OS)
Startup Time	Days / Weeks	Minutes	Minutes	Milliseconds
Isolation Level	Physical hardware	Hypervisor & VM	VM	Namespaces & cgroups
Complexity	Hardware procurement	Guest OS overhead	Abstracted APIs, pay-per-use	Declarative orchestration

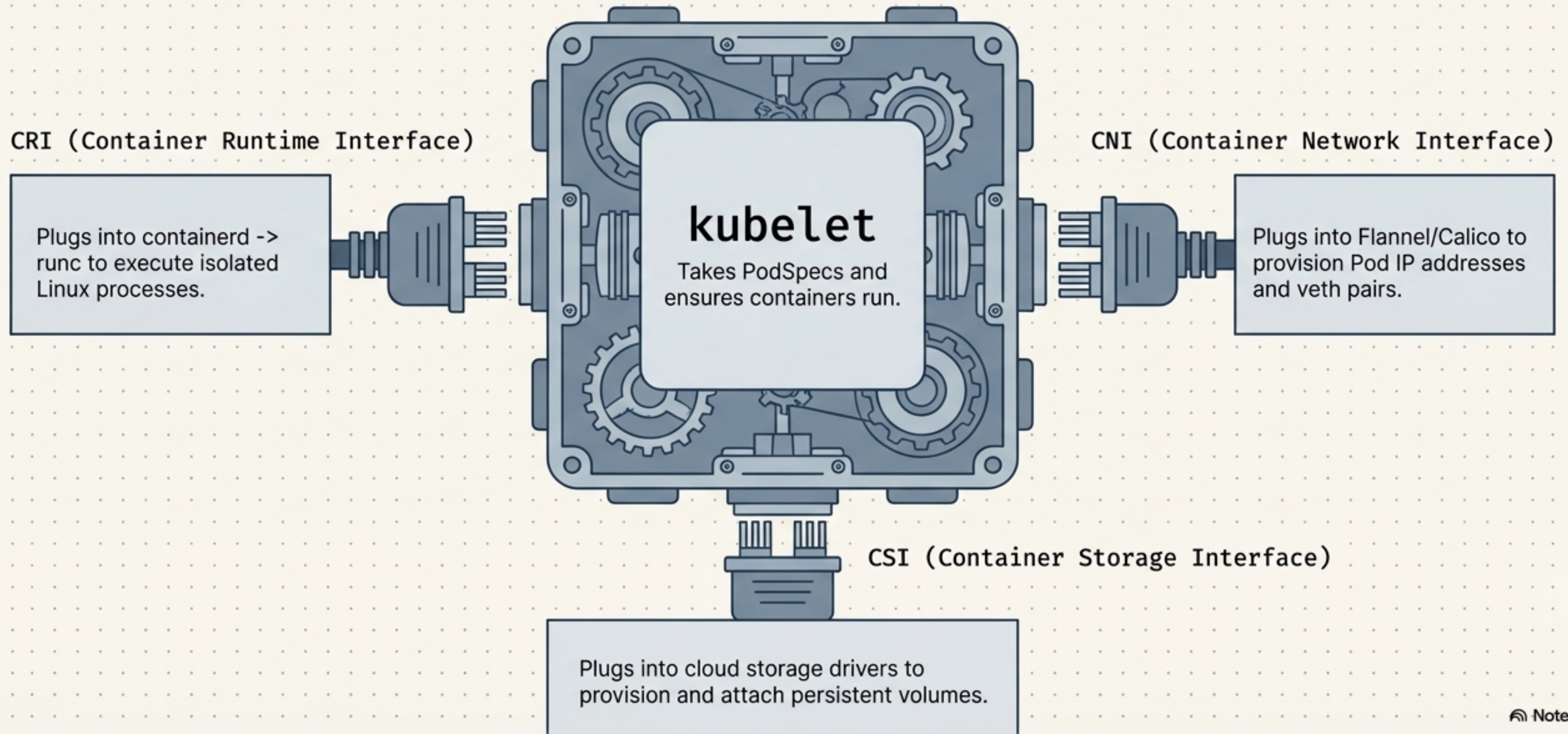
The Macro-Architecture: Control Plane and Worker Nodes



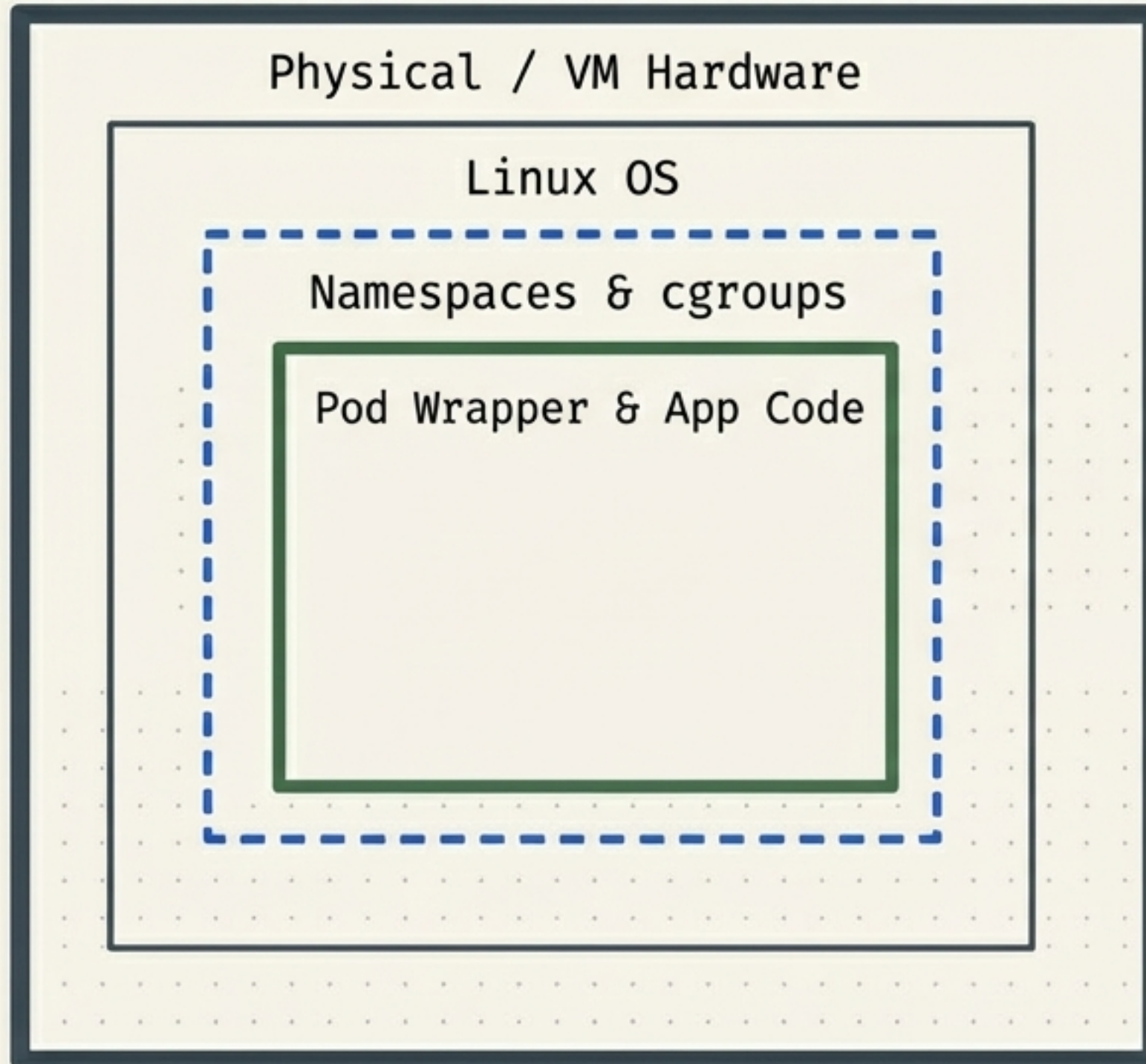
The API Server Security Checkpoint



The Worker Node Engine and Modular Interfaces



Anatomy of a Pod: The Smallest Deployable Unit



Physical/VM Layer

Provides raw CPU, RAM, and Disk.

cgroups

The resource slicer. Dictates the maximum CPU and Memory a process can consume.

Linux Namespaces

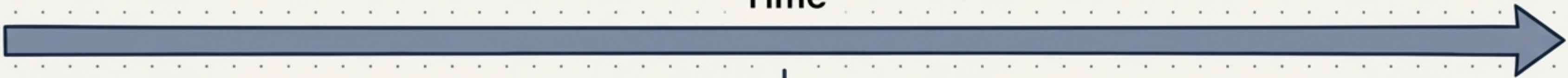
The isolation layer. Ensures an isolated view of the system (PID, NET, MNT).

The Pod Wrapper

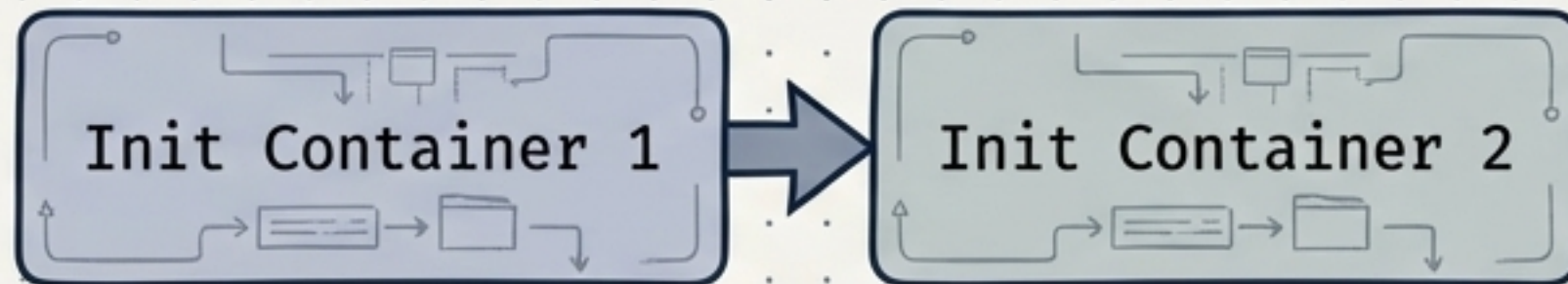
A logical Kubernetes abstraction wrapping containers that share Network Namespaces and volumes.

Multi-Container Execution Patterns

Time

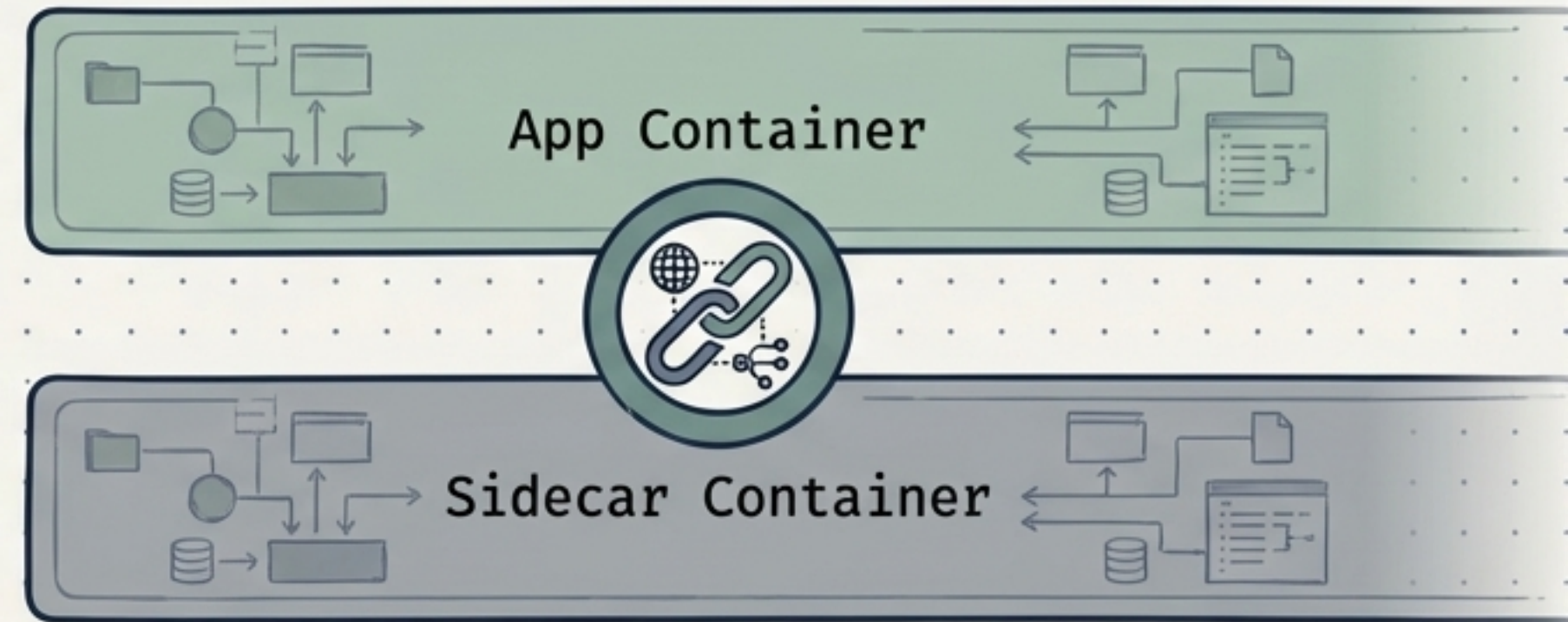


Sequential Execution



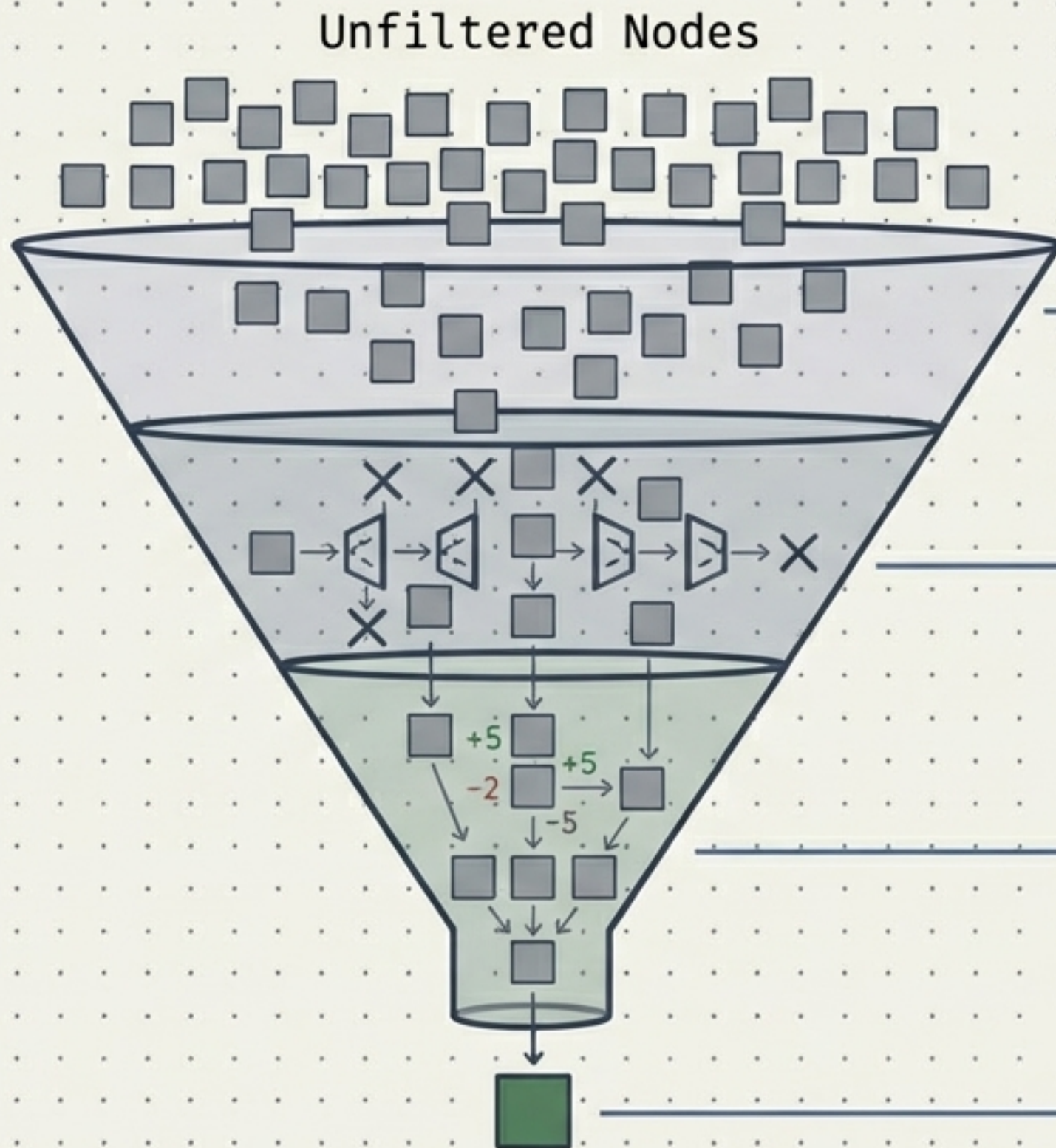
Init Containers run sequentially to completion before app containers start. Used for **setup logic** (e.g., **cloning a git repo**).

Parallel Execution



App and Sidecar Containers run in parallel. They share the exact same Network Namespace (communicating via localhost) and can mount the same shared volumes.

The Scheduling Funnel: Finding the Perfect Node



Step 1: Queueing - The Pod waits for assignment.

Step 2: Filtering (Feasibility) - Discards nodes that cannot run the Pod. Evaluates Node Selectors, Resource Limits, and Taints/Tolerations.

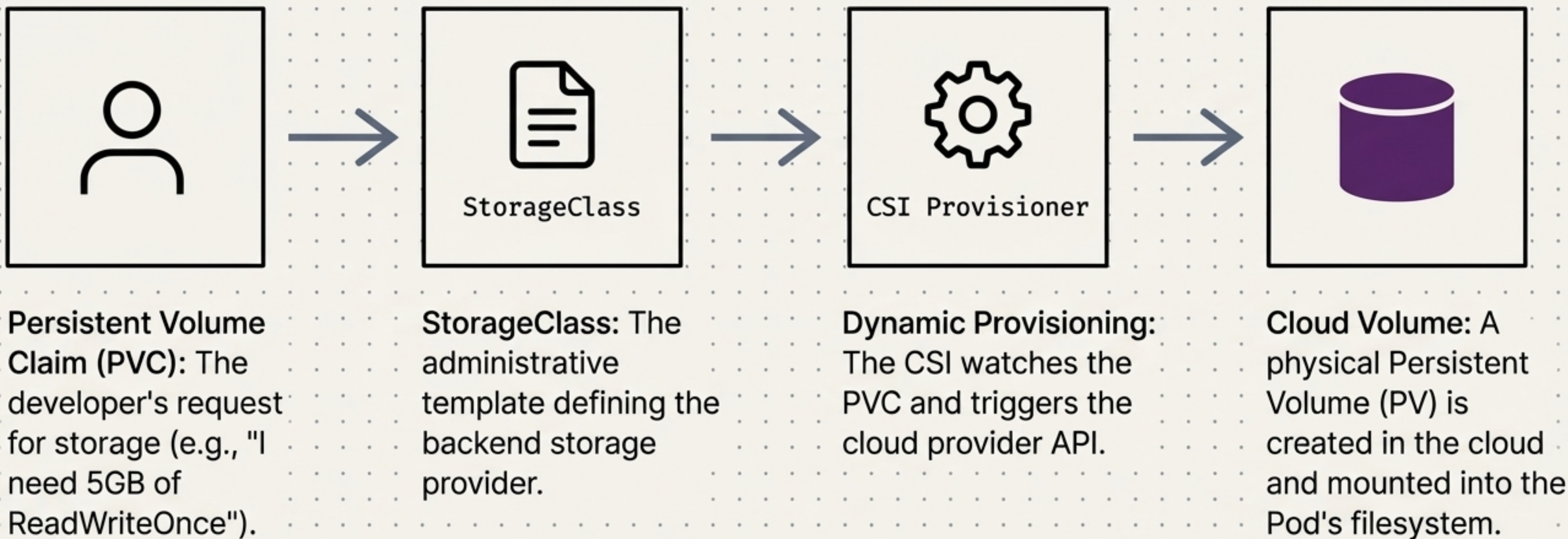
Step 3: Scoring (Optimization) - Ranks remaining nodes based on Node Affinity and Topology Spread.

Step 4: Binding - The highest-scoring node is written to etcd.

Choosing the Right Workload Controller

Deployments (Stateless)	StatefulSets (Stateful)	DaemonSets (Node-Level)
Scaling: ReplicaSets ensure N copies run.	Scaling: Strict, ordered deployment (0, 1, 2).	Scaling: Exactly one pod per node.
Naming: Random alphanumeric hashes.	Naming: Predictable, sticky identities (pod-0, pod-1).	Naming: Hash-based, bound to node lifespan.
Storage: Shared or ephemeral.	Storage: Unique VolumeClaimTemplates per pod.	Storage: Host-level mapping.
Use Case: Web frontends, stateless APIs.	Use Case: Databases (PostgreSQL), message queues.	Use Case: Log forwarders, CNI agents, monitoring.

The Persistent Storage Lifecycle

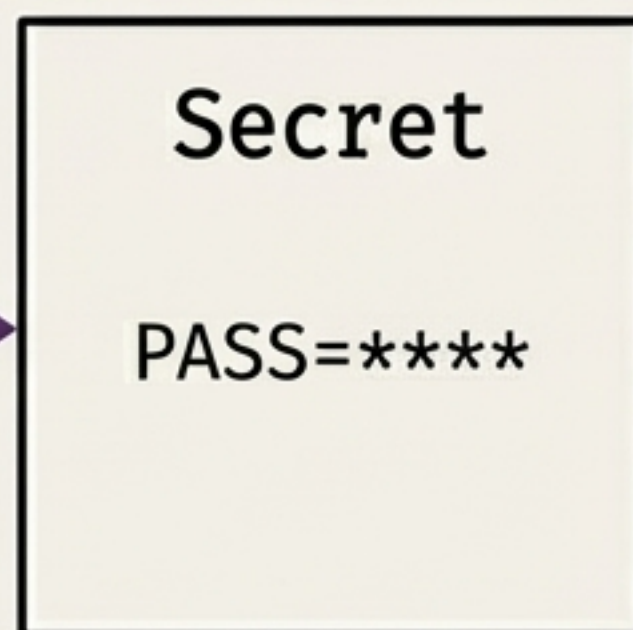


Decoupling Configuration and Credentials

The External Data

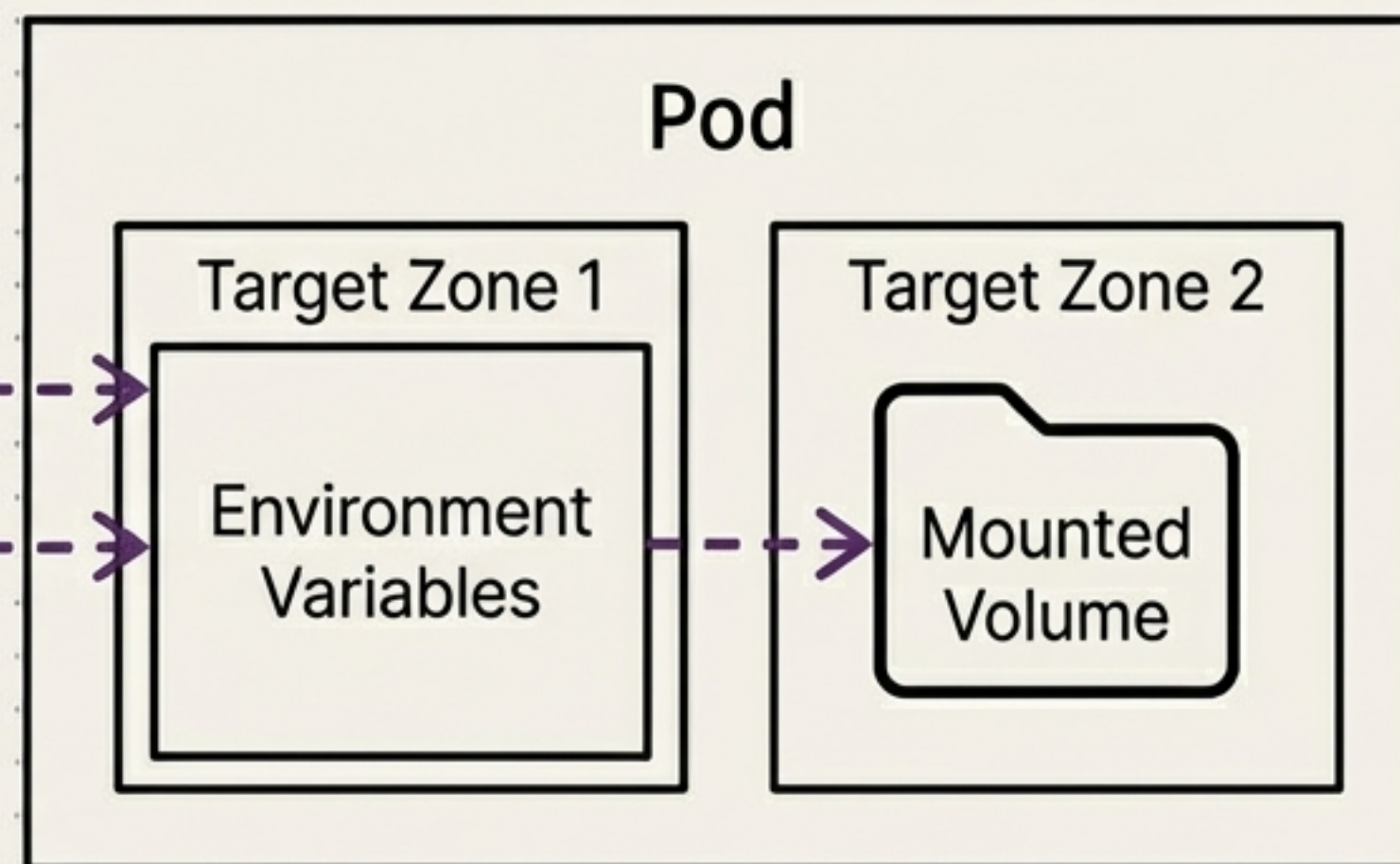


Stores non-confidential configuration (initialization scripts, URL strings).



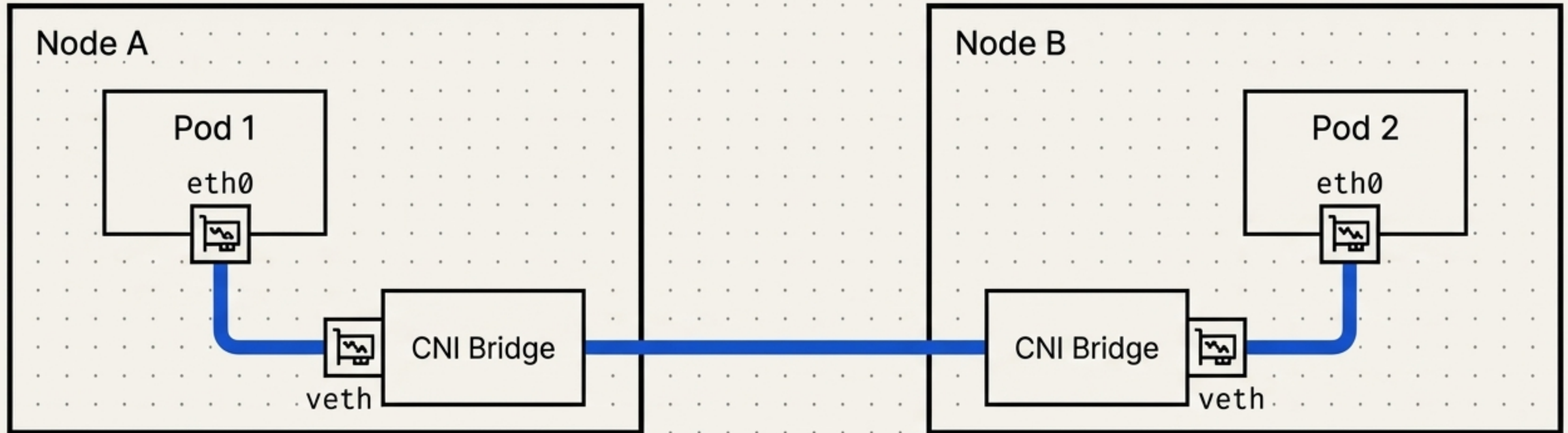
Stores sensitive data. Base64 encoded by default. Secure clusters enable Encryption at Rest in etcd.

The Workload



Both ConfigMaps and Secrets can be injected dynamically as environment variables or mounted directly as files.

Internal Pod-to-Pod Communication



The Flat Network

Every pod gets a unique IP address. Pods communicate across nodes without NAT.

The CNI Bridge

The CNI establishes a virtual ethernet pair (veth) connecting the isolated Pod network namespace to the host's root namespace.

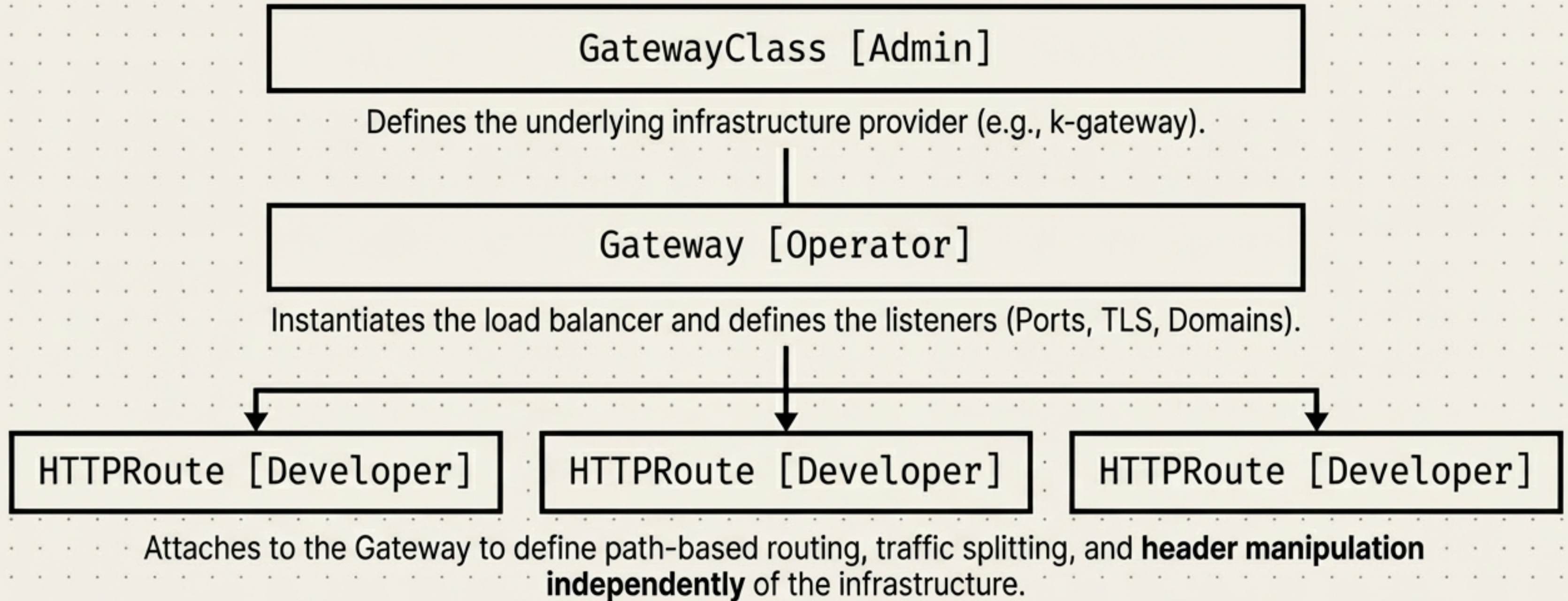
CoreDNS

Automatically maps static Service names (game-service.local) to these ephemeral Pod IPs.

Exposing Workloads: The Service Matrix

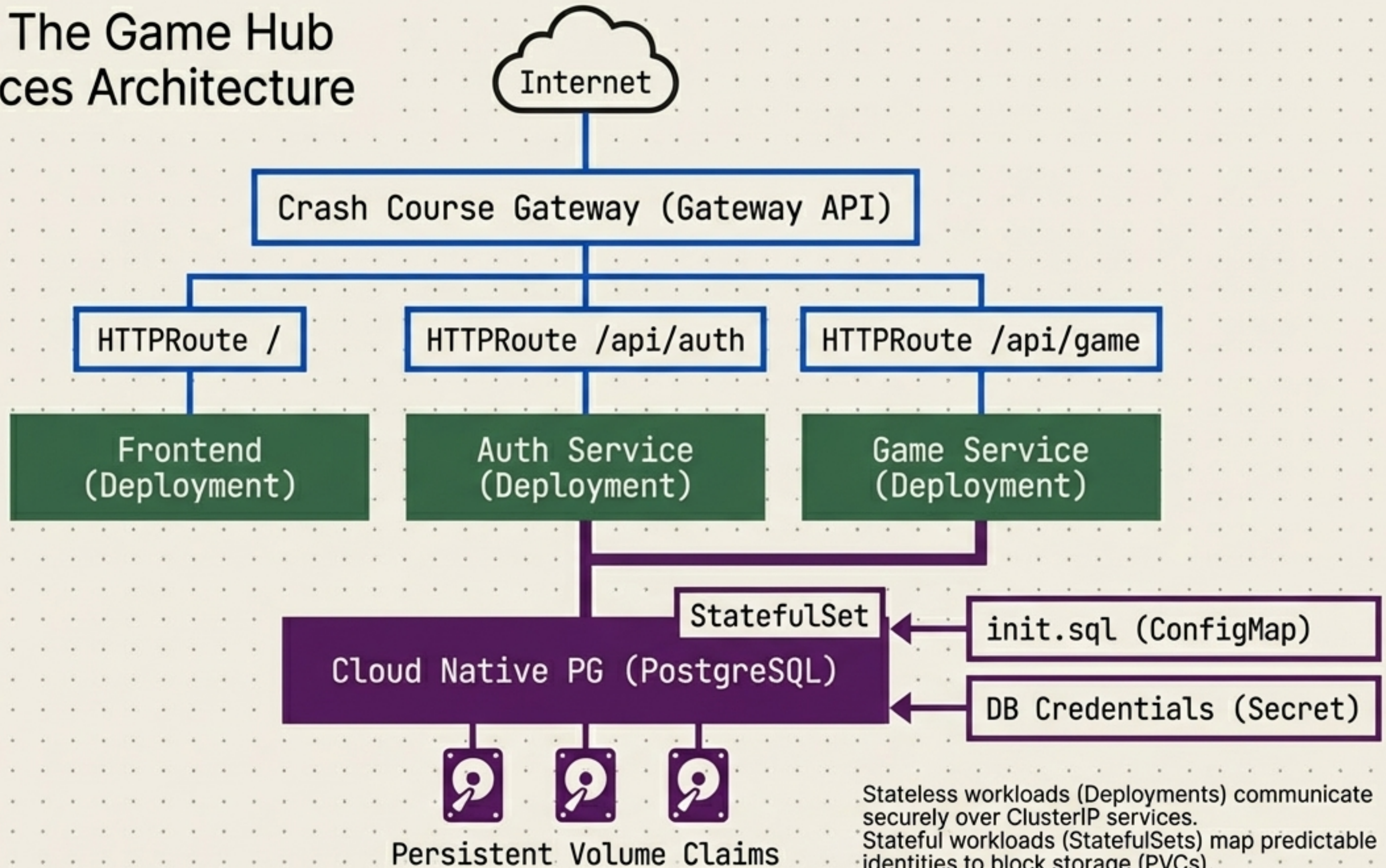
ClusterIP	NodePort	LoadBalancer	Headless Service
• Default internal service. • Provides a stable virtual IP that load-balances across healthy internal pods. • Not publicly accessible.	• Exposes the service on a static port (30000-32767) on every Worker Node. • Primarily used for bare-metal testing. • Not recommended for production external traffic.	• Triggers the Cloud Controller Manager. • Provisions a public, external IP (L4 load balancing). • Points directly to the underlying NodePort/ClusterIP.	• Defined explicitly by setting ClusterIP: None. • Bypasses load balancing entirely. • Returns the direct IP addresses of backing pods. • Essential for StatefulSets (e.g., reaching specific database replicas).

The Gateway API: Modern L7 Routing



The Problem with Ingress: Legacy Ingress relied heavily on non-standard, provider-specific annotations and lacked role-based separation.

Synthesis: The Game Hub Microservices Architecture



Stateless workloads (Deployments) communicate securely over ClusterIP services.
Stateful workloads (StatefulSets) map predictable identities to block storage (PVCs).
Secrets and ConfigMaps are injected at runtime.